

NNVISR: Bring Neural Network Video Interpolation and Super Resolution into Video Processing Framework

Yuan Tong
Mengshun Hu
Zheng Wang*

National Engineering Research Center for Multimedia Software, Hubei Key Laboratory of Multimedia and Network
Communication Engineering, School of Computer Science, Wuhan University
Wuhan, China
{yuan.tong,shunmh,wangzwhu}@whu.edu.cn

Abstract

We present NNVISR - an open-source filter plugin for the VapourSynth¹ video processing framework, which facilitates the application of neural networks for various kinds of video enhancing tasks, including denoising, super resolution, interpolation, and spatio-temporal super-resolution. NNVISR fills the gap between video enhancement neural networks and video processing pipelines, by accepting any network that enhances a group of frames, and handling all other network agnostic details during video processing. NNVISR is publicly released at <https://github.com/tongyuantongyu/vs-NNVISR>.

CCS Concepts

• **Human-centered computing** → **Visualization**; • **Hardware** → **Displays and imagers**; • **Computing methodologies** → **Computer vision**.

Keywords

open source, video super-resolution, video interpolation, spatio-temporal super-resolution

ACM Reference Format:

Yuan Tong, Mengshun Hu, and Zheng Wang. 2024. NNVISR: Bring Neural Network Video Interpolation and Super Resolution into Video Processing Framework. In *Proceedings of Proceedings of the 32nd ACM International Conference on Multimedia (MM '24)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3664647.3685522>

1 Introduction

Success on image super resolution [7, 13, 16] motivated the research of similar tasks on video. The nature of videos being a series of highly related images encouraged considering multiple images at the same time, which is the setting of video super resolution [2, 3, 15]. By having images at different time points there's a new time

*Corresponding author

¹<https://github.com/vapoursynth/vapoursynth>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '24, October 28-November 1, 2024, Melbourne, VIC, Australia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0686-8/24/10

<https://doi.org/10.1145/3664647.3685522>

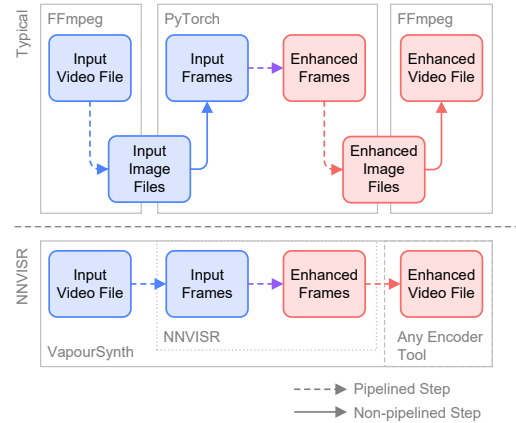


Figure 1: Comparison between typical and NNVISR video enhancement workflows. A non-pipelined step requires that the whole video, in this step's input form, be materialized on the storage at the same time.

axis for super resolution, or more commonly, video interpolation[1, 5, 11, 12]. Combining the two tasks together for better information sharing is the spatio-temporal super resolution [4, 9, 10, 17, 18] task.

When processing images, usually the whole input image is loaded into the computational memory. However, videos are indefinitely long sequences of images (frames), and can be extraordinarily large when in uncompressed form. Such intrinsic characteristic of video renders it unfeasible to process videos like images. In fact, uncompressed videos are so large that they can quickly fill up even non-volatile storage devices, let alone memory. To solve this problem, many video enhancement methods [1, 2, 8–10, 17, 18] actually works on a definite and relatively small number of consecutive frames, thus limiting the amount of data to be processed at a time, while still being able to explore the extra temporal relation between frames close to each other.

While such change to the task enabled applying deep learning method to video enhancing tasks, it left the ultimate task of enhancing videos half-done. Image enhancing tasks, on the contrast, have inputs and outputs of neural network models exactly be images and can be directly integrated into current applications. Most video enhancement models expect input to be a tuple of images, so to

utilize them, as showed in Fig. 1, one has to first convert the input video into image sequence, group the images into tuples, run the model inference on each tuple to obtain output image tuples, collect and merge output tuples into sequence, and finally convert the sequence back to video. This procedure is both inefficient and resource consuming. Some models' implementation do try to support processing video directly by using Python libraries like OpenCV, but OpenCV's video processing support is barely usable and far from the need of serious video processing, let alone performance. Moreover, by processing frame tuples instead of the whole video, video enhancement models avoided a non-trivial problem of detecting scene changes, across which exploring temporal relation usually resulted in artifacts instead of enhancements and should generally be avoided.

In developing a practical spatio-temporal video super resolution tool utilizing on our previous work CycMuNet[9] and YOGO[10], we realized that our tool can utilize not only our models, and even not only the spatio-temporal video super resolution models. Instead, any video enhancement model that designed to produces output frames based on a consecutive tuple of input frames can fit into our design. This motivated us to develop NNVISR, an open-source tool, to enable the usage of all kinds of video enhancement models into video processing workflow.

NNVISR is publicly released under the 3-Clause BSD License. The repository contains all the source code and detailed documentation including installation, compilation and model intergration guide. NNVISR can also be installed using the conda package manager, along with all necessary dependencies. NNVISR is designed as a filter plugin for the open source VapourSynth video processing framework. VapourSynth uses Python grammar to declare processing pipeline, and has mature support for video loading, traditional video processing and piping output to video encoders like FFmpeg and x264. NNVISR uses NVIDIA's TensorRT² framework to provide high performance inference of deep learning network. Therefore, NNVISR is suitable for both quick demo and large-scale deployment. To the best of our knowledge, NNVISR is the first video processing tool to support inter-frame video enhancement utilizing any neural network.

2 Design Overview

NNVISR provides a simple interface for both end users to process video, and implementors to add their models. On top of that, NNVISR also integrates well into the current VapourSynth ecosystem, to cooperate with other filters to form an advanced video processing pipeline.

2.1 End User Usage

The usage of NNVISR is simple. NNVISR follows the idiomatic design of VapourSynth filter, which provides a function that accepts an input video (conventionally called "clip" in VapourSynth) ,parameters to customize the behavior, and returns the processed video.

By setting scale factor and flag parameters to different values, NNVISR can be configured to perform various kinds of video enhancing tasks:

²<https://developer.nvidia.com/tensorrt>

```
1 from vapoursynth import core
2 # Loading video file as clip
3 clip = core.bs.VideoSource(source="input.mp4")
4 # Use CycMuNet to enhance the clip through NNVISR
5 clip = core.nnvisr.Super(clip,
6                           scale_factor=4,
7                           input_count=2,
8                           feature_count=64,
9                           extraction_layers=4,
10                          interpolation=True,
11                          extra_frame=True,
12                          double_frame=True,
13                          model="cycmunet/vimeo90k")
14 # Set the enhanced clip as output
15 clip.set_output()
```

Listing 1: Example script utilizing NNVISR

- Setting scale factor to 1 to perform video denoising.
- Setting scale factor > 1 to perform video super resolution.
- Setting scale factor to 1 and "Interpolation" flag to perform video interpolation.
- Setting scale factor to 1, "Interpolation" flag and "Double frame" flag to perform video interpolation along with denoising.
- Setting scale factor > 1, "Interpolation" flag and "Double frame" flag to perform spatial-temporal video resolution.

Usually real-world videos contains more than one scene while video enhancement models are only trained on a single scene. Therefore, if the scene boundary fallled into the input tuple, enhancement result is usually bad so this should be avoid. VapourSynth framework provides a standard way to signal the occurence of scene change, and NNVISR would respect the signal and make appropriate adjustment to avoid packing frames from different scenes into input tuple. User can choose the appropriate way to mark scene changes, either manually or by using a scene change detection filter. There are already multiple filters that detects scene change using traditional method, and employing a neural network to detect scene changes is also very easy in VapourSynth.

In addition to source code, we also published pre-built binaries for Windows and Linux as conda packages, so that users can easily install NNVISR without going through the environment configurations. A minimal example of VapourSynth processing script utilizing NNVISR is depicted in Listing 1.

2.2 Model Integration

NNVISR provide a simple yet expressive interface for video enhancement models to implement, in order to be utilized by NNVISR.

NNVISR focuses on supporting the kind of video enhancement neural models that explore temporal relationship between a definite number of consecutive frames, or a "frame group". We separate the whole process into two stages: the extract stage and the fusion stage. The extract stage operates on single frame and produces a pyramid of features in different spatial dimensions (Models that do not use feature pyramid can be seen as a pyramid of only one layer). The fusion stage mixes the features of all frames in a frame group to produce output frames.

For convenience, we define n to be the number of frames being **consumed** by the model for each inference run. Note that for

interpolation models, the last frame of a frame group is usually also the first frame of the next frame group to produce exactly 2 times the number frames, so in this case n is the number of input frames minus 1. Based on this definition, we introduced several flags to precisely describe model behavior:

- “Interpolation” flag means the model is doing video interpolation, and instruct NNVISR to produce output video in doubled framerate.
- “Extra frame” flag means the model accepts $n + 1$ frames rather than n .
- “Double frame” flag means the model produces $2n$ frames for each inference run, otherwise n . If “Double frame” flag is not set and “Interpolation” flag is set, then input frames are also used as output frames as is, and model outputs are all treated as intermediate frames.

To integrate neural model into NNVISR, model authors should provide their model definition in ONNX³ format, which is widely supported by deep learning frameworks. Notably, PyTorch has built-in support for automatically exporting model into ONNX format. Specially, we recognized broad usage of Deformable Convolution [6], especially the V2 [19] variant, among video enhancement models, mainly due to it’s ability to efficiently adapt to arbitrary object motion using its offset input, which is crucial in aligning object among frames and warping frames based on optical flow information. However, while Deformable Convolution plays an important role in video enhancement models, it does not have wide support, especially is not natively supported by TensorRT. TensorRT has a plugin interface to support custom network operators, so we implemented the V2 variant of Deformable Convolution for TensorRT. It is also compatible with PyTorch’s Deformable Convolution implementation provided in torchvision, so models trained using PyTorch can be integrated easily.

NNVISR supports input and output in both RGB and YUV pixel format, and implementors can choose to support either or both. RGB is the most common pixel format used in computer vision, while the YUV420 variant of YUV pixel format is the dominant format for video storage and processing. Output in YUV pixel format has two advantages: one being more efficient since YUV420 halves the spatial dimensions for 2 of its channels, therefore resulted in only half the amount of data compared to RGB; another being higher quality because conventional conversion from RGB to YUV420 has been shown to be suboptimal, especially for high dynamic range contents [14]. By training to output frame in YUV420 format neural networks can easily achieve better visual quality compared to conventional conversion method.

2.3 Internal Design

NNVISR internally performs the grouping of frames, taking account of n setting of model and scene change. For interpolation models, NNVISR uses a carefully designed frame layout in memory to reduce memory usage and memory movement as much as possible.

Interpolation models usually require $n + 1$ input frames, and for a batch of b , the naive way needs to provide $b \times (n + 1)$ frames to the model. However for a batch of b , the total number of related frames is actually $b \times n + 1$. This indicates that $b - 1$ frames are

	3	1	2	9	7	8	...
0	6	4	5	12	10	11	...

Figure 2: NNVISR frame memory layout (in column major) where $n = 3$ and $b = 2$. Numbers denote the frame index.

duplicated and it will take more space than necessary. The nature of batching requires inputs from different batches placed contiguously in memory. Rather than placing the frames in the naive order, NNVISR use an layout shown in Fig. 2. By pointing input addresses to frame 0, 1, 2 and 3 respectively, the following frames are 3, 4, 5 and 6 which are exactly the frames of next group. Meanwhile, each frame only appears once, so this layout eliminated the need to duplicate frames. To process the next batch, only the last frame need to be copied to the location of the second last frame (6 to 5 for the figure’s case).

NNVISR has a complete set of rules to arrange frames to both satisfy model input and respect scene change:

- Layout incoming frames in the way shown in Fig. 2, if there’s no scene change mark on frames.
- Upon seeing scene change signal, shift the grouping window forward, until either the last frame in window is the frame before scene change; or the first frame in window is the first frame of current scene.
- Duplicate last frame until the grouping window is filled.

This set of rules makes NNVISR try its best to feed model with distinct frames for the last group, so the model can perceive most temporal information.

3 Empirical Results

NNVISR is designed to employ a wide range of models to accomplish the video enhancement task. End users can enhance their videos using the model of their choice with a significant speed uplift, and researchers can easily integrate their models for downstream application consumption. To experimentally validate the effectiveness of NNVISR, we integrated 5 well-known models into NNVISR and conducted an comparison on performance, as shown in Table 1. Model files for these models are also released with NNVISR.

3.1 Performance Experiment

To showcase the performance boost of NNVISR, we measured the total time used in enhancing a 400-frame clip with each network, in both original implementation in PyTorch and using NNVISR. We selected several typical video resolutions for measurement to better reflect real world performance. For PyTorch implementation, input frames are pre-loaded into memory; For NNVISR, input frames are decoded during the enhancement process. This is a handicap for NNVISR, but VapourSynth is able to schedule the decoding and enhancement to run concurrently, so the performance impact is negligible.

Table 1 shows the performance comparison between PyTorch implementation and NNVISR, measured by output FPS (frame per second). All results are measured on an NVIDIA RTX 4090 GPU, using PyTorch 2.3.0 for the original (PyTorch) implementation, and TensorRT 8.6.1 for our (NNVISR) implementation. All models except

³<https://github.com/onnx/onnx>

Table 1: Performance Comparison

Model	Input Frames	Scale Ratio	Output Dim.	Org. (fps)	Ours (fps)	Speed Ratio
Zooming- SlowMo [17]	2	4	1080p	12.69	25.44	2.00
			2160p	2.44	6.35	2.60
	4		1080p	9.88	27.47	2.78
			2160p	2.08	6.88	3.31
TMNet [18]	2	4	1080p	11.61	22.77	1.96
			2160p	2.24	5.60	2.50
	4		1080p	9.88	24.79	2.51
			2160p	1.91	6.19	3.24
Cyc- MuNet [9]	2	2	1080p	1.61	5.53	3.43
			2160p	OOM	1.41	-
	4		1080p	2.34	8.23	3.52
			2160p	OOM	2.06	-
VideoINR [4]	2	2	720p	3.20	5.01	1.57
		4	1080p	OOM	5.06	-
YOGO [10]	2	4	1080p	15.31	32.17	2.10
			2160p	2.88	7.88	2.74
	4		1080p	13.24	33.62	2.54
			2160p	2.47	8.39	3.40

VideoINR are using half precision computation for both implementations, while VideoINR is using single precision. The reason is explained in Section 3.2.

3.2 Result Analysis

Table 1 shows that NNVISR consistently accelerates different models, on different resolutions, by a significant ratio of about 2 or higher. The only exception is VideoINR[4] on which NNVISR only achieves an acceleration ratio of 1.57. VideoINR uses grid sample instead of deformable convolution for feature warping. Grid sample uses absolute coordinate to represent source location, therefore is highly sensitive to numerical imprecision. We found that we have to run VideoINR under single precision instead of half precision to ensure correct result. TensorRT is less optimized for single precision so the ratio is lower. This also causes both implementations to quickly run out of available GPU memory even under a relatively small resolution.

We would like to point out that real world time savings on enhancing a video is well beyond the measurements shown in Table 1. Time spent on encoding and decoding frames to and from image format can take much longer than actual enhancement work. Whereas for NNVISR, as showed in Fig. 1, output can be directly piped into video encoding tools to form a fully streamlined video enhancement workflow, achieving zero extra wall-clock time for decoding or encoding.

4 Conclusion and Future Work

We have publicly released NNVISR, a versatile video processing tool that supports arbitrary models and various kinds of video enhancing tasks. Moreover, it's high performance, well documented, easy to

use and integrates well into current video processing toolchain. In this report, we introduced the design of NNVISR as well as the considerations behind. We believe NNVISR can bridge the gap between research and application, therefore boost the adoption of video enhancement research achievements into downstream industrial application scenarios.

While the current iteration of NNVISR supports many models, new models sometimes utilize custom operators and support more complicated interpolation mode. In future work, we plan to support more custom operators used in newer models, and extending frame interpolation support beyond doubling frames.

Acknowledgments

This work was supported by the Hubei Key R&D Project (2022BAA033) and the National Natural Science Foundation of China (62171325).

References

- [1] Wenbo Bao, Wei-Sheng Lai, Chao Ma, Xiaoyun Zhang, Zhiyong Gao, and Ming-Hsuan Yang. 2019. Depth-Aware Video Frame Interpolation. In *CVPR*.
- [2] Jose Caballero, Christian Ledig, Andrew Aitken, Alejandro Acosta, Johannes Totz, Zehan Wang, and Wenzhe Shi. 2017. Real-Time Video Super-Resolution With Spatio-Temporal Networks and Motion Compensation. In *CVPR*.
- [3] Kelvin C.K. Chan, Xintao Wang, Ke Yu, Chao Dong, and Chen Change Loy. 2021. BasicVSR: The Search for Essential Components in Video Super-Resolution and Beyond. In *CVPR*. 4947–4956.
- [4] Zeyuan Chen, Yinbo Chen, Jingwen Liu, Xingqian Xu, Vedit Goel, Zhangyang Wang, Humphrey Shi, and Xiaolong Wang. 2022. VideoINR: Learning Video Implicit Neural Representation for Continuous Space-Time Super-Resolution. In *CVPR*. 2047–2057.
- [5] Myungsub Choi, Heewon Kim, Bohyung Han, Ning Xu, and Kyoung Mu Lee. 2020. Channel Attention Is All You Need for Video Frame Interpolation. *AAAI* 34, 07 (Apr. 2020), 10663–10671.
- [6] Jifeng Dai, Haozhi Qi, Yuwen Xiong, Yi Li, Guodong Zhang, Han Hu, and Yichen Wei. 2017. Deformable Convolutional Networks. In *ICCV*.
- [7] Chao Dong, Chen Change Loy, Kaiming He, and Xiaoou Tang. 2016. Image Super-Resolution Using Deep Convolutional Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38, 2 (2016), 295–307.
- [8] Muhammad Haris, Greg Shakhnarovich, and Norimichi Ukita. 2020. Space-Time-Aware Multi-Resolution Video Enhancement. In *CVPR*.
- [9] Mengshun Hu, Kui Jiang, Liang Liao, Jing Xiao, Junjun Jiang, and Zheng Wang. 2022. Spatial-Temporal Space Hand-in-Hand: Spatial-Temporal Video Super-Resolution via Cycle-Projected Mutual Learning. In *CVPR*. 3574–3583.
- [10] Mengshun Hu, Kui Jiang, Zhixiang Nie, and Zheng Wang. 2022. You Only Align Once: Bidirectional Interaction for Spatial-Temporal Video Super-Resolution. In *ACM MM (Lisboa, Portugal) (MM '22)*. Association for Computing Machinery, New York, NY, USA, 847–855.
- [11] Zhewei Huang, Tianyuan Zhang, Wen Heng, Boxin Shi, and Shuchang Zhou. 2022. Real-Time Intermediate Flow Estimation for Video Frame Interpolation. In *ECCV*.
- [12] Huaizu Jiang, Deqing Sun, Varun Jampani, Ming-Hsuan Yang, Erik Learned-Miller, and Jan Kautz. 2018. Super SloMo: High Quality Estimation of Multiple Intermediate Frames for Video Interpolation. In *CVPR*.
- [13] Christian Ledig, Lucas Theis, Ferenc Huszar, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, and Wenzhe Shi. 2017. Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network. In *CVPR*.
- [14] Jacob Ström, Jonatan Samuelsson, and Kristofer Dovstam. 2016. Luma adjustment for high dynamic range video. In *DCC*. IEEE, 319–328.
- [15] Yapeng Tian, Yulun Zhang, Yun Fu, and Chenliang Xu. 2020. TDAN: Temporally-Deformable Alignment Network for Video Super-Resolution. In *CVPR*.
- [16] Xintao Wang, Ke Yu, Shixiang Wu, Jinjin Gu, Yihao Liu, Chao Dong, Yu Qiao, and Chen Change Loy. 2018. ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks. In *ECCV Workshop*.
- [17] Xiaoyu Xiang, Yapeng Tian, Yulun Zhang, Yun Fu, Jan P. Allebach, and Chenliang Xu. 2020. Zooming Slow-Mo: Fast and Accurate One-Stage Space-Time Video Super-Resolution. In *CVPR*.
- [18] Gang Xu, Jun Xu, Zhen Li, Liang Wang, Xing Sun, and Ming-Ming Cheng. 2021. Temporal Modulation Network for Controllable Space-Time Video Super-Resolution. In *CVPR*. 6388–6397.
- [19] Xizhou Zhu, Han Hu, Stephen Lin, and Jifeng Dai. 2019. Deformable ConvNets V2: More Deformable, Better Results. In *CVPR*.